Welcome to the official developer and system documentation for **HR Assist**. This document provides an in-depth look at the architecture, feature set, technical execution, database schemas, API surfaces, and deployment configurations of the HR Assist platform.

1. System Vision & Overview

HR Assist is an enterprise-grade conversational AI platform designed to bridge the gap between complex company policy documentation and employee information access. By utilizing a **Retrieval-Augmented Generation (RAG)** pipeline, HR Assist serves as a single source of truth, delivering accurate, grounded, and role-appropriate answers to user questions while strictly preventing hallucinations and unauthorized data leakage.

Key Value Propositions

- **Role-Based Access Control (RBAC):** Restricts information and system capabilities dynamically based on user identity (Admin, HR, Staff, Guest).
- **Dual-Mode Inference:** Provides cloud-speed operations using the cloud-based **Groq API** with an automatic offline fallback to **Ollama** running Microsoft's local **Phi-3** model.
- **Intelligent Memory Integration:** Supports persistent session context matching roles, with durable long-term storage for HR personnel and standard in-memory short-term context for standard staff.
- **Dynamic Document Management:** Empowers HR representatives to upload, delete, and re-index policy PDFs on the fly with real-time vector database reconstruction.
- **Complete Administrative Portal:** Provides an administrative dashboard to manage user accounts, oversee candidate recruitment registries, and perform critical management operations with double-confirmation security verification.

2. Technical Stack

HR Assist is built using a modern, lightweight, and robust stack suited for rapid local execution and standard cloud containment.

```
graph TD subgraph Frontend [Glassmorphic Client] UI[Vanilla HTML5 / CSS3 / JS] Marked[Marked.js Markdown Parser] end subgraph Backend [Flask Web Server] Flask[app.py Core Server] DataHandler[data_handler.py pandas handler] end subgraph RAG [Retrieval-Augmented Generation] LC[LangChain Framework] Embed[all-MiniLM-L6-v2 Embeddings] Chroma[ChromaDB Vector Store] end subgraph AI [Inference Engines] Groq[Groq API: LLaMA 3.1 8B Cloud] Ollama[Ollama: Phi-3 Local] end subgraph Storage [Data Store] Excel[staffrecruitment.xlsx] Users[users.json] Memories[hr_memories.json] end UI <-->|HTTP REST / REST API| Flask Flask <--> DataHandler Flask <--> LC LC <--> Embed LC <--> Chroma LC <--> Groq LC <--> Ollama DataHandler <--> Excel Flask <--> Users Flask <--> Memories
```

Components Table

Component	Technology	Role / Purpose
Frontend UI	HTML5, Vanilla CSS3 (Glassmorphism), Vanilla JavaScript	Responsive client portal, chat bubbles, responsive panels, modal logic, Markdown rendering.
Backend Framework	Python / Flask	Web server, request validation, API routes, RBAC token mapping, session memory persistence.
RAG Pipeline	LangChain / HuggingFace Embeddings	Document ingestion, smart text segmentation, semantic vectorization (`all-MiniLM-L6-v2`), similarity retrieval.
Vector Database	ChromaDB	Highly optimized embedded database to store and perform similarity search on text-chunk vectors.
Cloud LLM	Groq Cloud API (`llama-3.1-8b-instant`)	Ultrafast, deterministic cloud text generation (1-3 seconds response).
Local LLM	Ollama (`phi-3`)	Fully offline local LLM execution fallback to maintain service during cloud outages or network disconnection.
File Handler	Pandas	Reads, parses, and dynamic search filters of standard recruitment registries (`staffrecruitment.xlsx`).
Deployment Engine	Docker / Gunicorn	Containerization, and production-grade WSGI web server management.

3. System Architecture & Request Lifecycle

When an end-user interacts with the chat interface, the query undergoes structured validation, semantic retrieval, role routing, and token checks.

```
sequenceDiagram autonumber actor User as User Browser participant Flask as Flask Server participant DB as ChromaDB participant Excel as Excel Store participant LLM as LLM Engine (Groq/Ollama) User->>Flask: POST /chat { question, mode, role, session_id } (Auth Token in Headers) Note over Flask: 1. Validate JSON Payload<br/>2. Match Role from Token Database rect rgb(30, 41, 59) Note over Flask: RAG Ingestion & Retrieval Flask->>DB: Semantic Similarity Search (Retrieve top 5 chunks) DB-->>Flask: Context Chunks (Text content & metadata) end alt User is HR or Admin & Web Search is On Note over Flask: Live Web Crawling Flask->>Flask: Execute DuckDuckGo Web Search API end alt Query involves Recruitment Candidates Note over Flask: Database Integration Flask->>Excel: Fetch Candidate List (via data_handler.py) Excel-->>Flask: Structured Excel Rows end Note over Flask: Compile Chat History Context (Short/Long term) Note over Flask: Build Unified ChatPromptTemplate Flask->>LLM: Dispatch Compiled Prompt (Context + Question + History) LLM-->>Flask: Generated Response string Note over Flask: Append exchange to matching history structure Flask->>User: JSON Response { answer, mode, time_taken }
```

4. Feature Implementation Details

4.1 Hybrid Memory Systems

To support natural conversational flows, HR Assist segments chat memory context according to security levels:

1. **HR Long-Term Memory:** Dynamically loaded and stored on disk inside `hr_memories.json`. This memory persists through server restarts, session expirations, and page refreshes.
2. **Staff/Guest Short-Term Memory:** Stored in-memory inside the Flask process (`short_term_memories` dictionary). Retains context during active chat sessions but does not persist to disk, optimizing database footprint and privacy.
3. **Unified Memory Parser:** Chat history is formatted into standard human-AI exchanges and fed into the prompt template dynamically:

```
def format_chat_history(history, max_turns=6): recent_messages = history[-(max_turns * 2):] formatted = [] for msg in recent_messages: role = "Human" if msg["role"] == "user" else "AI" formatted.append(f"{role}: {msg['content']}") return "\n".join(formatted) if formatted else "No previous conversation history."
```

4.2 Role-Based Access Control (RBAC)

User permissions are checked at the API layer using Bearer-like token validation:

- **Admin Role:** Full system configuration dashboard. User account CRUD operations, dynamic viewing of candidate database, configuration control.
- **HR Role:** Policy document management panel (upload files, retire files, trigger automatic vector db rebuilds), active DuckDuckGo internet searches, full system Q&A.

- **Staff Role:** Restricted strictly to questioning internal PDFs. Cannot upload files, cannot utilize web search, restricted to client chat view.
- **Guest Role:** Restricted strictly to standard policies (recruitment policy, company policy, leave policy). Prompts outside this domain are intercepted and rejected gracefully.

4.3 Policy upload & Dynamic DB Rebuilds

HR representatives can expand the chatbot's domain by uploading new policy PDFs. When a document is uploaded via `/upload_policy`:

1. The file is sanitized using `werkzeug.utils.secure_filename`.
2. The file is saved directly to the application workspace directory.
3. The server appends the file to the dynamic configuration tracker (`metadata_pdfs.json`).
4. An automated routine clears the existing Chroma DB instance, loads all registered PDFs, splits them into `RecursiveCharacterTextSplitter` chunks, re-calculates vectors via Hugging Face embeddings, and rebuilds the search retriever on the fly.
5. Deleting a policy via `/delete_policy` triggers the reverse, unindexing and deleting files cleanly from disk.

4.4 Admin Management Suite & Security Verification

The Admin Console provides a complete control panel for administrators to manage users and access recruitment logs, wrapped in an elegant dark-theme control board. To secure critical endpoints:

- **Double-Confirmation on Deletion:** Deleting user accounts requires a two-step prompt. After clicking the red "Delete" button and confirming the warning popup, the admin is presented with a **Verify Admin Identity** modal asking for their password passkey.
- **Password Verification Modal:** Identity verification is processed dynamically through backend database verification, comparing the entered password directly with the secure stored records of the logged-in administrator.

5. Storage & Database Schema

The platform maintains its databases in clean JSON structures and structured Excel logs to facilitate lightweight reading and manual management.

5.1 User Accounts Database (`users.json`)

Stores registered credentials, RBAC roles, and authentication tokens:

```
[ { "username": "admin", "password": "admin@2025", "role": "admin", "token": "ADMIN_TOKEN" }, { "username": "Hr1", "password": "hr@2025", "role": "hr", "token": "HR1_TOKEN" }, { "username": "Staff1", "password": "staff1@2025", "role": "staff", "token": "STAFF1_TOKEN" } ]
```


5.2 Conversational Memory Store (`hr_memories.json`)

Durable dictionary mapping HR users to their complete message logs:


```
{ "HR1": [ { "role": "user", "content": "Who got shortlisted for the developer role?" }, { "role": "assistant", "content": "According to the staff recruitment logs, the shortlisted candidates are:\n- Akash Baburaj\n- Emily Watson" } ] }
```


5.3 Recruitment Ledger (`staffrecruitment.xlsx`)

Maintained as an Excel sheet inside the root directory. Fields read by the RAG context generator and Admin Console include:

- `App. ID` (Application ID)
- `Full Name`
- `Gender`
- `Department`
- `Position Applied`
- `Experience (Yrs)`
- `Status` (e.g., Shortlisted, Selected, Rejected, On Hold)
- `Remarks`

6. API Reference

All requests must contain appropriate `Content-Type: application/json` headers. Role-protected routes require a valid `Authorization` token header.

6.1 Authentication API

`POST /login`

Authenticates user credentials and returns session permissions and matching token.

- **Request Body:**

```
{ "username": "Hr1", "password": "hr@2025" }
```

- **Response (Success - 200 OK):**

```
{ "role": "hr", "token": "HR1_TOKEN" }
```

6.2 Chat API

`POST /chat`

Submits user questions, queries vector database, appends memory/web searches/excel logs based on role, and queries selected LLM engine.

- **Request Body:**

```
{ "question": "What is the policy on maternity leave?", "mode": "groq", "role": "staff", "internet": false, "session_id": "STAFF_SESSION_XYZ" }
```

- **Headers:**

```
Authorization: STAFF1_TOKEN
```

- **Response (Success - 200 OK):**

```
{ "answer": "According to page 12 of the HR Policy manual, maternity leave entitlement is...", "mode": "groq", "time_taken": 1.45 }
```

6.3 Memory API

`GET /chat_history`

Retrieves chat context history for the active session or logged-in user.

- **Request Parameters:** `session_id` (string, optional)
- **Response (200 OK):**

```
{ "history": [ { "role": "user", "content": "Hi" }, { "role": "assistant", "content": "Hello! How can I assist you with company policies today?" } ] }
```

`POST /clear_history`

Removes conversational history logs permanently.

- **Request Body:**

```
{ "session_id": "STAFF_SESSION_XYZ" }
```

- **Response (200 OK):**

```
{ "status": "success", "message": "Chat history cleared successfully." }
```

6.4 Dynamic Document Management API

`GET /active_policies`

Lists all currently active and indexed PDF documents inside the vector retriever.

- **Response (200 OK):**

```
{ "pdf_list": ["hr_policy.pdf", "staffrecruitment.pdf", "new_safety_guidelines.pdf"] }
```

`POST /upload_policy`

Uploads a new policy document and triggers automatic vector store reconstruction.

- **Headers:** `Authorization: HR1_TOKEN`
- **Request Payload:** `multipart/form-data` containing a `file` parameter.
- **Response (200 OK):**

```
{ "status": "success", "message": "Successfully uploaded and indexed 'new_safety_guidelines.pdf'!",
"pdf_list": ["hr_policy.pdf", "staffrecruitment.pdf", "new_safety_guidelines.pdf"] }
```

`POST /delete_policy`

Unindexes and deletes a dynamically uploaded PDF policy file.

- **Headers:** `Authorization: HR1_TOKEN`
- **Request Body:**

```
{ "filename": "new_safety_guidelines.pdf" }
```

- **Response (200 OK):**

```
{ "status": "success", "message": "Successfully retired and un-indexed 'new_safety_guidelines.pdf'!",
"pdf_list": ["hr_policy.pdf", "staffrecruitment.pdf"] }
```

6.5 Administrative API

`GET /api/admin/users`

Lists all user credentials, roles, and authorization tokens.

- **Headers:** `Authorization: ADMIN_TOKEN`
- **Response (200 OK):**

```
{ "users": [ { "username": "admin", "role": "admin", "token": "ADMIN_TOKEN", "password": "..." } ] }
```

`POST /api/admin/users`

Registers a new user inside the credential registry.

- **Headers:** `Authorization: ADMIN_TOKEN`
- **Request Body:**

```
{ "username": "newuser", "password": "user@2025", "role": "staff" }
```

- **Response (200 OK):**

```
{ "status": "success", "message": "User 'newuser' added successfully." }
```

`PUT /api/admin/users/<string:old_username>`

Updates password, username, or role for an existing account.

- **Headers:** `Authorization: ADMIN_TOKEN`
- **Request Body:**

```
{ "username": "updatedusername", "password": "newpassword@2025", "role": "hr" }
```

- **Response (200 OK):**

```
{ "status": "success", "message": "User updated successfully." }
```

`DELETE /api/admin/users/<string:username>`

Deletes user from registry database. Admins are blocked from self-deletion.

- **Headers:** `Authorization: ADMIN_TOKEN`
- **Response (200 OK):**

```
{ "status": "success", "message": "User 'username' deleted successfully." }
```

`GET /api/admin/recruitment`

Loads active candidate grids from Excel log sheet directly.

- **Headers:** `Authorization: ADMIN_TOKEN`

---

7. Developer Operations & Setup Guide

7.1 Dynamic Environment Variables Configuration (`.env`)

Create a `.env` file in the root workspace directory before starting the application:

```
# Groq API cloud configuration GROQ_API_KEY=gsk_your_groq_cloud_api_key_goes_here # Local Ollama host definition OLLAMA_HOST=http://localhost:11434
```

7.2 Local Installation & Execution

Follow these steps to run the application in a virtual Python environment under Windows:

```
# 1. Establish python environment inside root folder python -m venv venv # 2. Activate Python sandbox environment .\venv\Scripts\activate # 3. Install required library specifications pip install -r requirements.txt # 4. Launch Flask Web application python app.py
```

After starting the backend, open your browser and navigate to **`http://localhost:7860`**.

7.3 Exposing Application via Ngrok Tunnel

To make the application publicly available for teammates or mobile testing:

```
# Fire up public tunnel directing traffic to local Flask web server port ngrok http 7860
```

This generates a secure, public HTTPS link (e.g., `https://exporter-gusto-broom.ngrok-free.dev`) that tunnels directly to your local instance.

7.4 Dockerized Deployment

To package the workspace for cloud spaces like Hugging Face Spaces:

```
# Build the standardized docker image docker build -t hr-assist . # Launch local docker container running isolated environment docker run -p 7860:7860 --env-file .env hr-assist
```

8. Frontend Interface Design & Aesthetics

The HR Assist front-end utilizes a highly refined **Glassmorphism** visual system that presents a sleek, modern user experience:

- **Visual Components:** Uses background image integration coupled with backdrop blurs (`backdrop-filter: blur(16px)`), harmonized translucent panels, dynamic gradients, rounded cards, and elegant input fields.
- **Role Styles:**
- Standard User: Friendly purple and blue theme designed for cozy, focused chatting.
- Admin Portal: Automatic transition into a clean, dark administrative interface, utilizing deep borders, red action tables, status badges, and highlighted search and CRUD logs to emphasize system operations.
- **Responsive Layout:** Automatically collapses into a single-column view with a sliding navigation drawer on smaller viewports, ensuring full layout fidelity on mobile phones and tablets.